

Computer Science I

Week 1, Lecture 3

Exercise Session

Exercise 1 - Celsius, Fahrenheit, And Kelvin Table

Write a program similar to the Fahrenheit-to-Celsius table from lecture. This time, print a table with three columns:

Celsius	Fahrenheit	Kelvin
---------	------------	--------

The Celsius values should go from 0 to 100, increasing by 10.

Example beginning of the output:

0	32.0	273.1
10	50.0	283.1
20	68.0	293.1

Exercise 2 - Squares, Cubes, And Running Sums

Write a program that prints a table for the numbers from 1 to 10.

For each number n , print:

- n ;
- $n * n$;
- $n * n * n$;
- the sum of all integers from 1 to n .

For example, the first rows should be:

1	1	1	1
2	4	8	3
3	9	27	6
4	16	64	10

This exercise is still close to the Fahrenheit table, but the program now has to remember something from one loop iteration to the next.

Exercise 3 - Count All Characters Until EOF

You have already seen programs like `io_v1.c` and `io_v2.c` (see Blackboard), where `getchar()` reads characters from standard input.

Now write a program that reads all characters until the input ends and prints:

- the total number of characters read;
- the final value returned by `getchar()` when the input ends.

Hints

Declare the variable that stores the result of `getchar()` as:

```
int c;
```

Question: why do we use `int` here, even though `getchar()` reads one character at a time? `getchar()` must be able to return all possible character values, but it must also be able to return one special value when the input is finished. That special value is called `EOF`, and it is defined in `stdio.h`.

You can use `EOF` directly in the condition of the loop:

```
while (c != EOF) {  
    ...  
}
```

Example output format:

```
Number of characters read: 8  
The End Of File (EOF) character has been reached and getchar() returned -1
```

The exact number printed for `EOF` can depend on the system, but it is commonly `-1`.

Exercise 4 - Count Only Uppercase E

Now modify the previous exercise.

Instead of counting all characters, count only how many times the uppercase character `'E'` appears.

Before writing the program, look at this example of `if`, `else if`, and `else`:

```
if (x > 0) {  
    printf("positive\n");  
} else if (x == 0) {  
    printf("zero\n");  
} else {  
    printf("negative\n");  
}
```

The conditions are tested in order. The first true block runs, and the rest are skipped. An `else if` means “if the previous condition was false, try this new condition.” The final `else` runs only if all previous conditions were false.

For this exercise you only need an `if`; you do not need an `else`:

```
if (c == 'E') {  
    ...  
}
```

Example output format:

```
Number of 'E' characters read: 3  
The End Of File (EOF) character has been reached and getchar() returned -1
```

Exercise 5 - A Small Character Counter

The next natural idea is to count many different characters.

Write a program that reads all input until `EOF` and counts:

- how many times `'A'` appears;

- how many times 'E' appears;
- how many newline characters '\n' appear;
- how many other characters appear.

Hint: Use the `if`, `else if`, `else` pattern introduced in Exercise 4.

Each character should increase exactly one counter.

Example input:

```
EAGLE
bee
```

Expected counts:

```
A: 1
E: 2
newline: 2
other: 5
```

To obtain `newline: 2` interactively, press Enter after `EAGLE`, press Enter after `bee`, then press Ctrl-D on the empty line.